Tugas 3
Implementasi AI Pemasaran - Video Generator to Drive

Arief Rachman Apriansyah - 0110224166
Grup 3
TEKNIK INFORMATIKA 2024

Sekolah Tinggi Teknologi Terpadu Nurul Fikri
Depok
2026

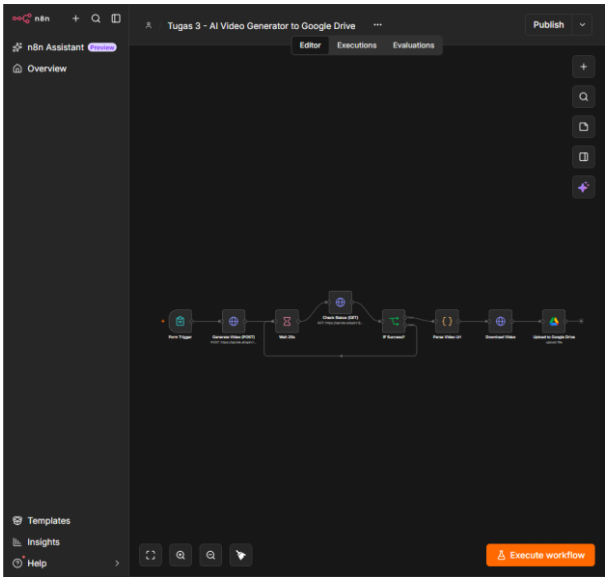
A. Pendahuluan

Tugas ini meminta kita merakit sebuah workflow di n8n yang mampu mengubah prompt teks menjadi video, lalu menyimpan hasilnya ke Google Drive tanpa campur tangan manual. Kalau dipikir-pikir, pekerjaan semacam ini dekat sekali dengan kebutuhan pemasaran. Tim konten sering membutuhkan video pendek untuk promosi, sementara proses menunggu render, mengunduh file, dan mengunggahnya ulang satu per satu jelas memakan waktu.

Laporan ini menjelaskan bagaimana kelima bagian tugas saya kerjakan, mulai dari formulir input sampai file video mendarat di folder Drive. Setiap tahap saya sertakan tangkapan layarnya sebagai bukti. Semua bagian berjalan sesuai yang diminta.

B. Gambaran Umum Workflow

Workflow saya beri nama **Tugas 3 - AI Video Generator to Google Drive**. Isinya delapan node yang tersambung berurutan: Form Trigger, Generate Video (POST), Wait 20s, Check Status (GET), IF Success?, Parse Video Url, Download Video, dan Upload to Google Drive. Yang membuat alurnya tidak lurus biasa adalah jalur kembali dari node IF ke node Wait. Jalur itulah yang membentuk siklus pengecekan, dan akan dibahas di bagian berikutnya.



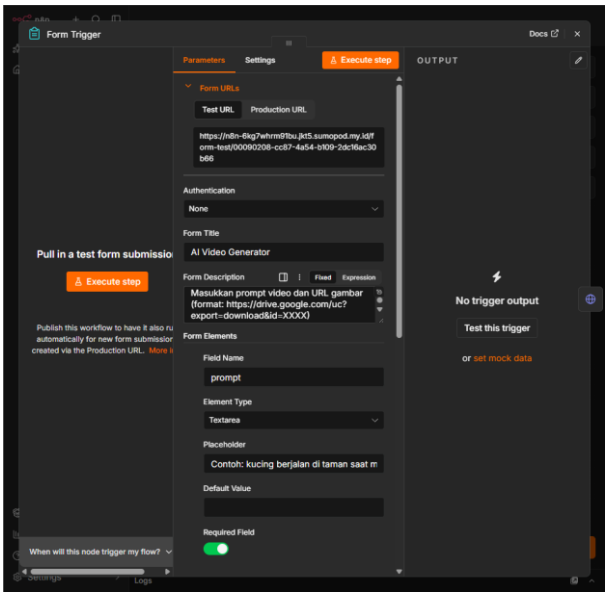
Gambar 1. Tampilan keseluruhan workflow di editor n8n

C. Pembahasan Tiap Tahap

1. Form Trigger dan permintaan POST ke API video

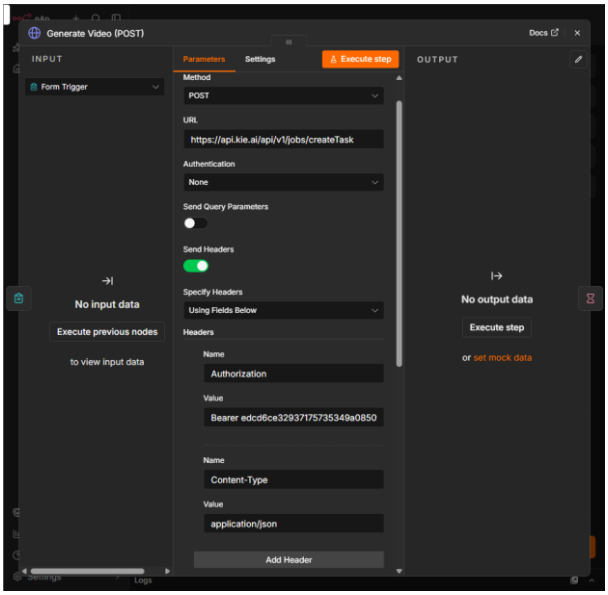
Titik awalnya adalah node **Form Trigger**. Formulir saya beri judul "AI Video Generator" dengan satu kolom bernama prompt bertipe Textarea, dan kolom ini saya wajibkan diisi. Alasannya sederhana: tanpa prompt, tidak ada yang bisa dikirim ke API, jadi lebih baik

formulir menolak input kosong sejak awal. Placeholder-nya berisi contoh kalimat tentang kucing berjalan di taman supaya pengguna langsung paham gaya prompt yang diharapkan. Deskripsi formulir juga memuat petunjuk format URL gambar dari Google Drive.



Gambar 2. Konfigurasi node Form Trigger

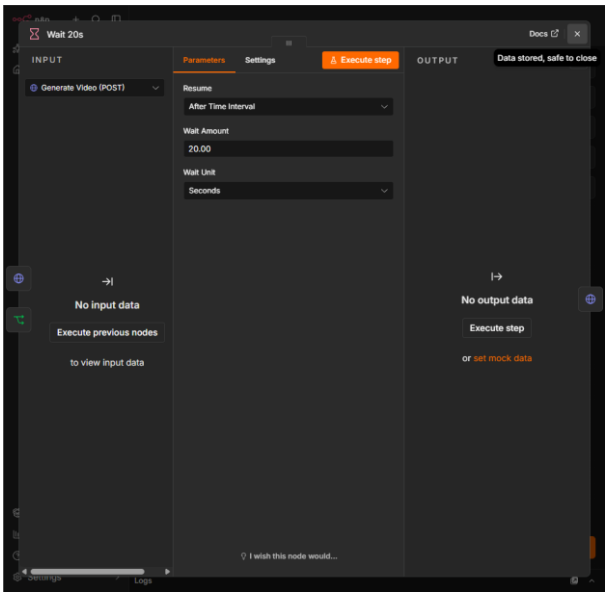
Data dari formulir diteruskan ke node **Generate Video (POST)**. Metodenya POST ke endpoint https://api.kie.ai/api/v1/jobs/createTask. Autentikasi tidak memakai fitur bawaan n8n; token dikirim lewat header Authorization berformat Bearer, ditemani header Content-Type: application/json karena isi permintaannya berupa JSON. Respons dari endpoint ini bukan video, melainkan sebuah taskId. Detail kecil ini ternyata menentukan seluruh rancangan workflow setelahnya.



Gambar 3. Node Generate Video (POST) dengan header Authorization dan Content-Type

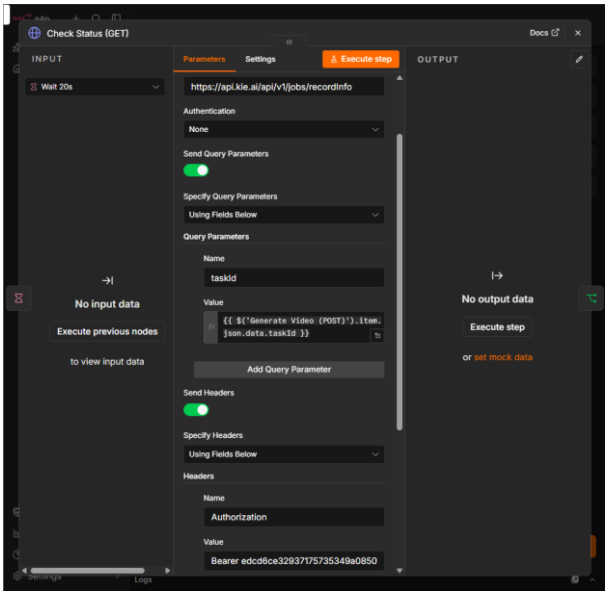
2. Node Wait 20 detik dalam siklus loop

Karena API hanya mengembalikan taskId, video belum tentu siap saat kita bertanya. Pembuatan video oleh AI butuh waktu, dan server perlu diberi jeda. Di sinilah node **Wait 20s** dipakai. Konfigurasinya: Resume diatur After Time Interval, Wait Amount 20, dan Wait Unit Seconds.



Gambar 4. Node Wait dengan jeda 20 detik

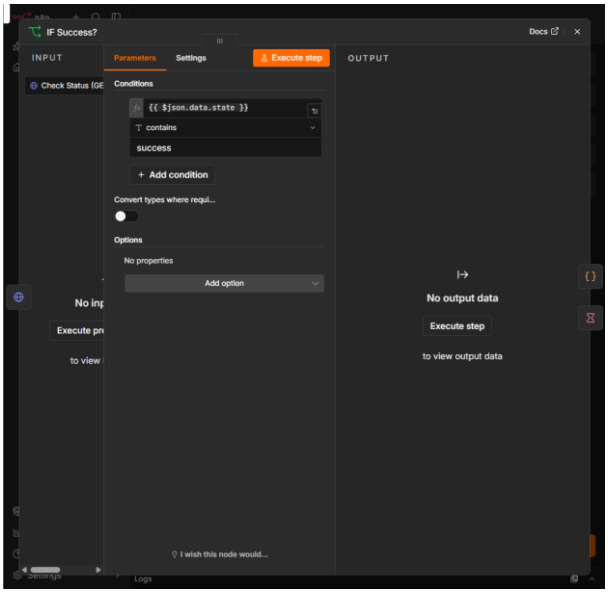
Setelah menunggu, giliran node **Check Status (GET)** bekerja. Ia memanggil https://api.kie.ai/api/v1/jobs/recordInfo dengan query parameter taskId, yang nilainya diambil dari node sebelumnya lewat ekspresi {{ $('Generate Video (POST)').item.json.data.taskId }}. Header Authorization kembali dipasang di sini.



Gambar 5. Node Check Status (GET) beserta query parameter taskId

Hasil pengecekan lalu dinilai oleh node **IF Success?**. Kondisinya membaca {{ $json.data.state }} dan memeriksa apakah nilainya mengandung kata "success". Kalau ya, alur berlanjut ke tahap

pengambilan URL. Kalau belum, alur kembali ke node Wait dan menunggu 20 detik lagi, begitu seterusnya sampai status berhasil.



Gambar 6. Kondisi pada node IF Success?

Ada satu hal yang saya sadari setelah workflow jadi. Cabang "false" menampung semua status selain sukses, termasuk status gagal. Artinya, kalau suatu hari proses di server gagal, workflow akan terus berputar tanpa ujung. Untuk keperluan tugas ini hal itu tidak menjadi masalah, tetapi untuk penggunaan sungguhan saya akan menambahkan cabang khusus untuk status gagal, atau membatasi jumlah putaran.

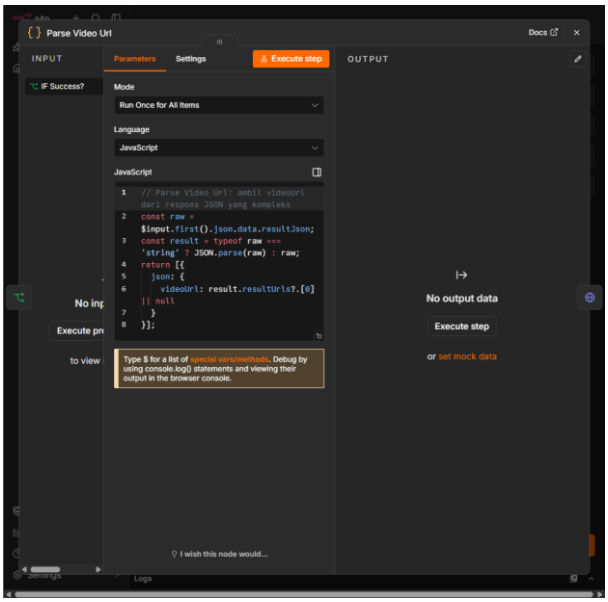
3. Code Node untuk mengekstrak videoUrl

Ketika status sudah sukses, URL video tidak tersedia langsung di permukaan respons. Ia terkubur di dalam data.resultJson, dan isinya berupa string JSON yang masih harus diurai. Jadi ada dua lapis yang perlu dibuka. Untuk itu saya menulis skrip berikut pada node **Parse Video Url** (mode Run Once for All Items, bahasa JavaScript):

```
// Parse Video Url: ambil videoUrl dari respons JSON yang kompleks
const raw = $input.first().json.data.resultJson;
const result = typeof raw === 'string' ? JSON.parse(raw) : raw;
return [{
  json: {
    videoUrl: result.resultUrls?.[0] || null
  }
}];
```

Baris pertama mengambil resultJson dari item pertama. Baris kedua mengecek tipenya: kalau berupa string, ia diurai dengan JSON.parse, kalau sudah berupa objek, dipakai apa adanya. Pengecekan ini berguna karena format respons bisa berbeda. Terakhir, result.resultUrls?.[0]

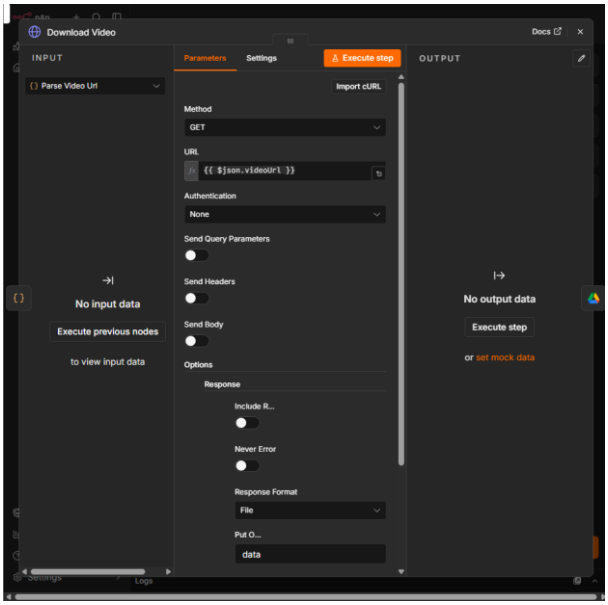
mengambil URL pertama dari daftar hasil. Tanda ?. mencegah skrip error ketika resultUrls ternyata tidak ada, dan nilai cadangan null memastikan node tetap mengeluarkan field videoUrl dalam keadaan apa pun.



Gambar 7. Skrip JavaScript pada Code Node Parse Video Url

4. HTTP Request untuk mengunduh video

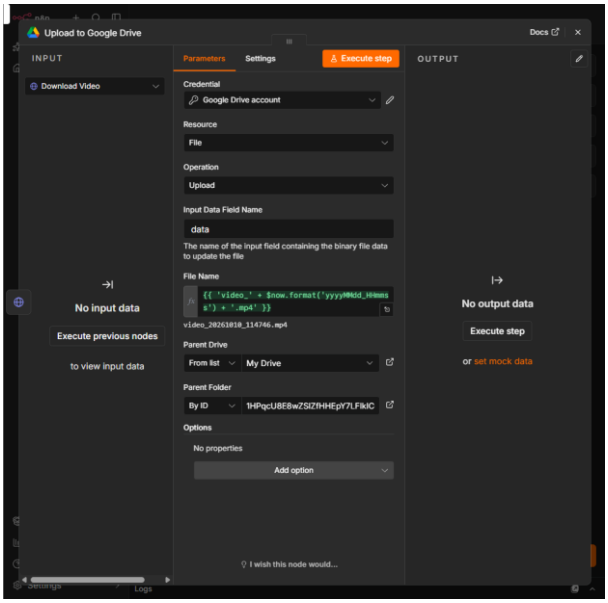
Dengan videoUrl di tangan, node **Download Video** mengunduh filenya. Metode yang dipakai GET, dengan URL berupa ekspresi {{ $json.videoUrl }}. Tidak ada autentikasi, query, header, maupun body, karena tautan hasil video bisa diakses langsung. Bagian paling krusial ada di opsi Response: Response Format saya atur ke **File**, dan hasilnya ditaruh pada field bernama data. Tanpa pengaturan ini, n8n akan memperlakukan isi video sebagai teks dan hasilnya tidak bisa diunggah. Nama field data ini juga yang nanti dipanggil oleh node Google Drive.



Gambar 8. Konfigurasi node Download Video dengan Response Format File

5. Upload otomatis ke Google Drive

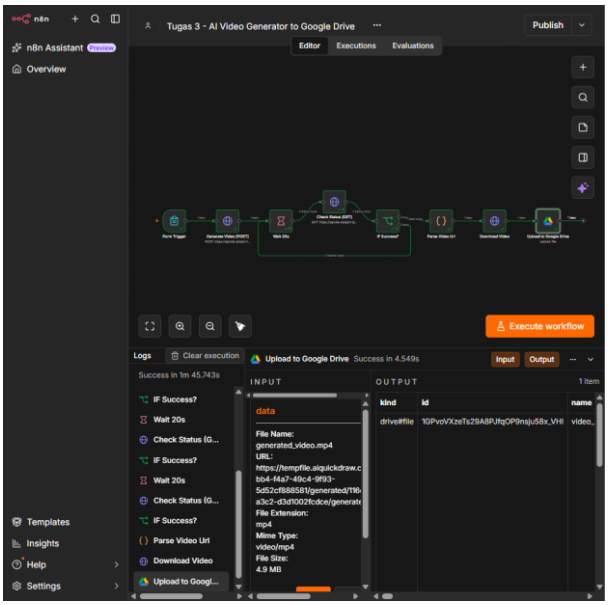
Tahap terakhir dikerjakan node **Upload to Google Drive** dengan kredensial akun Google Drive saya. Resource diisi File dan Operation diisi Upload. Input Data Field Name saya tulis data, sama dengan field keluaran node sebelumnya. Nama file dibuat dinamis lewat ekspresi {{ 'video_' + $now.format('yyyyMMdd_HHmmss') + '.mp4' }}, sehingga tiap video punya nama unik berdasarkan waktu dan tidak saling menimpa. Lokasi tujuannya adalah My Drive, dengan folder yang ditentukan lewat ID (By ID).



Gambar 9. Konfigurasi node Upload to Google Drive

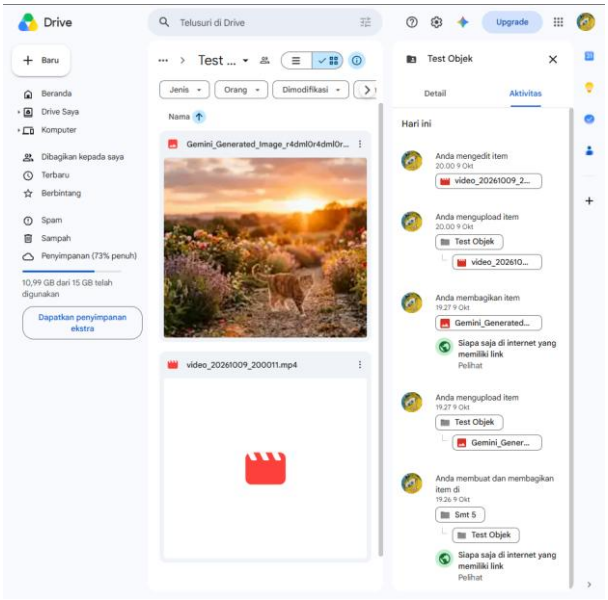
D. Hasil Pengujian

Setelah semua node terkonfigurasi, workflow saya jalankan dari formulir. Seluruh node berstatus sukses (ditandai centang hijau), dengan total durasi eksekusi 1 menit 45,743 detik. Pada jalur loop, label di garis koneksi menunjukkan empat item melewati node Wait dan Check Status, dengan tiga di antaranya kembali lewat jalur pengulangan sebelum akhirnya status sukses. Dengan kata lain, video belum siap pada tiga pengecekan pertama, dan loop menjalankan tugasnya dengan benar. Node terakhir, Upload to Google Drive, selesai dalam 4,549 detik. Datanya menunjukkan file generated_video.mp4 bertipe video/mp4 berukuran 4,9 MB, dan keluarannya bertanda drive#file yang menandakan file sudah tercatat di Drive.



Gambar 10. Hasil eksekusi workflow dan output node Upload to Google Drive

Bukti terakhir ada di Google Drive sendiri. Di folder tujuan "Test Objek" muncul file video_20261009_200011.mp4, dan panel aktivitas di sisi kanan mencatat bahwa file itu diunggah pada pukul 20.00 tanggal 9 Oktober. Pola nama file-nya sama persis dengan format yang saya tulis di ekspresi, jadi jelas ini hasil dari workflow, bukan unggahan manual. Di folder yang sama terlihat pula sebuah gambar kucing berjalan di taman saat matahari terbenam, sejalan dengan contoh prompt pada formulir.



Gambar 11. File video hasil workflow di folder Google Drive

E. Refleksi

Pelajaran terbesar dari tugas ini buat saya adalah soal sifat asinkron API. Awalnya saya mengira satu permintaan POST akan langsung membawa video pulang. Kenyataannya, yang kita terima hanya tiket antrean, dan sisanya adalah urusan menunggu serta bertanya berulang kali. Pola Wait, Check, lalu IF ini rupanya umum dipakai untuk layanan AI generatif, bukan hanya video.

Dua hal lain yang saya pelajari: tipe data biner harus diperlakukan khusus saat mengunduh, dan data hasil API sering datang berlapis sehingga butuh skrip kecil untuk membukanya. Kekurangan yang masih ada adalah penanganan status gagal pada loop, seperti dibahas di bagian C.2. Soal keamanan, token API sebaiknya disimpan sebagai credential n8n dan tidak ditulis langsung di header.

Dari sisi pemasaran, manfaatnya cukup nyata. Satu formulir sederhana sudah cukup bagi anggota tim non-teknis untuk memesan video, dan hasilnya otomatis tersimpan rapi di folder bersama.

F. Kesimpulan

Kelima bagian tugas berhasil dikerjakan. Formulir menerima prompt dan mengirimkannya lewat POST ke API video. Node Wait 20 detik dalam siklus loop memberi waktu server memproses video sebelum statusnya dicek ulang. Code Node mengekstrak `videoUrl` dari respons JSON berlapis, HTTP Request mengunduh videonya sebagai data biner, dan node

Google Drive mengunggahnya ke folder tujuan. Bukti pada Gambar 10 dan Gambar 11 memastikan alur ini berjalan dari ujung ke ujung.

G. Tautan

Link Folder Google Drive : https://drive.google.com/drive/folders/1HPqcU8E8wZSlZfHHEpY7LFlkICezip6C?usp=sharing
Link Github JSON Workflow : https://github.com/Riansyah96/NFA_Implementasi-AI-Pemasaran/blob/main/Pertemuan_2_Tugas3_AI_Video_to_GDrive.json